AI in Chemistry

DACHA HITESH KUMAR U00344406
THIPIREDDY TARUN REDDY U00345391

Abstract

Artificial Intelligence in Chemistry is rapidly growing and a tremendous impact is visible on the end product. In nature, there are various energies available and to detect what kind of chemical/energy it is based on molecular formula is a tedious and accurate task to do.
In our case study, we will be writing the model to predict the ground state energy based on its molecular formula.

Keywords: Ground Energy, Neural network, machine learning chemistry, molecular structure.

Table of Index

I. Introduction to AI
II. AI in Chemistry
III. Data Preprocessing
IV. Model Development
V. Conclusion
VI. Reference

I. Introduction to AI

Artificial Intelligence is a branch of computer science that deals with the improving capability of a machine to produce the output rationally from the given input. The ultimate goal of Artificial Intelligence is to develop a system that performs the task that in other cases requires human intelligence.

In our day-to-day life, we are nowadays using lots of AI applications like a prediction in stocks, search, virtual assistants, chatbots, etc.

These applications are developing its popularity and the outcomes are getting better and better day by day because of the subfield of Artificial Intelligence which is Machine Learning and Deep Learning.

Machine Learning and Deep Learning algorithm learns and develop their capability in the same way as humans do. Humans learn and improve skills by constantly doing trial and error on a certain task. Similarly, Machine Learning and Deep Learning algorithm will in some specific manner does trial and error on the available input data and train themselves to react and give output for different types of inputs.

Artificial Intelligence development is growing at a rapid pace and is significantly contributing to different industries. Different Industries are constantly working on this area to crack the algorithm and develop the system to improve their business. AI is still in the innovation face, only developers and those industry experts who know AI are integrating AI into their business but as soon as the benefits of AI in terms of economic are visible, integration of AI will become at jet speed.

II. AI in Chemistry

In the research and development area of chemistry, AI is playing a vital role in today's time. Different Neural Networks and Machine Learning algorithms are making scientist task easier and helping them to develop molecular formulas and different vaccines faster and with more accuracy.

For our case study project, we are using the dataset that contains the molecular structure of ground state energies. The dataset contains 16242 unique molecule data records and each record contains1277 features of the molecule.

Using this dataset, we will train a model to predict the energy of the molecule. We will be training different models like KNN, Random Forest, and XG Boost on the dataset.

III. Data Preprocessing

In artificial intelligence, data is the most important part of any model building. Without data, there is no model and even if the data is not in the correct format model is of no use. Before working on the model, data processing is to be done with accuracy.

We will first import all the required libraries for model development as shown in the below figure.

```
import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV

from sklearn.base import BaseEstimator, TransformerMixin, RegressorMixin

from sklearn.ensemble import RandomForestRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.linear_model import Ridge, Lasso
from lightgbm.sklearn import LGBMRegressor
from xgboost.sklearn import XGBRegressor
from sklearn.metrics import mean_squared_error

from mlxtend.regressor import StackingCVRegressor
from sklearn.model_selection import cross_val_score
from sklearn.metrics import make_scorer
```

Figure 1

Now we will import the data from a CSV file using pandas.

```
df = pd.read_csv('roboBohr.csv')
```

```
df.head()
```

	0	1	2	3	4	5	6	7	8	9	...	1265	1266
0	73.516695	17.817765	12.469551	12.458130	12.454607	12.447345	12.433065	12.426926	12.387474	12.365984	...	0.5	0.0
1	73.516695	20.649126	18.527789	17.891535	17.887995	17.871731	17.852586	17.729842	15.864270	15.227643	...	0.0	0.0
2	73.516695	17.830377	12.512263	12.404775	12.394493	12.391564	12.324461	12.238106	10.423249	8.698826	...	0.0	0.0
3	73.516695	17.875810	17.871259	17.862402	17.850920	17.850440	12.558105	12.557645	12.517583	12.444141	...	0.0	0.0
4	73.516695	17.883818	17.868256	17.864221	17.818540	12.508657	12.490519	12.450098	10.597068	10.595914	...	0.0	0.0

5 rows × 1275 columns

Figure 2

Now we will check the shape of the dataset and if there are any null values present in it.

The shape of the data is (16242,1278) and there are no null values found in the dataset as shown in figure 3.

```
print(df.shape)
```

```
(16242, 1278)
```

```
df.isnull().sum().sum()
```

```
0
```

Figure 3

```
df.columns
```

```
Index(['Unnamed: 0', '0', '1', '2', '3', '4', '5', '6', '7', '8',
       ...
       '1267', '1268', '1269', '1270', '1271', '1272', '1273', '1274',
       'pubchem_id', 'Eat'],
      dtype='object', length=1278)
```

```
df = df.drop(['Unnamed: 0', 'pubchem_id'], axis = 1)
```

Figure 4

Out of 1278 columns, unnamed:0 and pubchem_id is something that will be not required while building the model as unnamed: 0 is the serial number and pubchem_id is the id given to each molecular structure.

```
df.Eat.describe()
```

```
count    16242.000000
mean       -11.178969
std          3.659133
min        -23.245373
25%        -13.475805
50%        -10.835211
75%         -8.623903
max         -0.789513
Name: Eat, dtype: float64
```

Figure 5

Eat is a target variable here and figure 5 describes how equally it is distributed. In figure 6 also the plot diagram shows the distribution of Eat variable values.

```
sns.distplot(df['Eat'], kde=True)
plt.xlabel('Energy')
plt.ylabel('Frequency')
plt.title('Energy Distribution');
```

```
/usr/local/lib/python3.8/dist-packages/seaborn/distributions.py:
  warnings.warn(msg, FutureWarning)
```

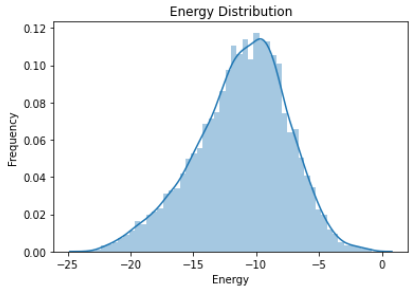

Figure 6

Now we will be using PCA (Principal Component Analysis) to find the feature that is having high correlation with the target variable as compared to other dataset.

```
df_new = pd.read_csv('coulombL.csv',header=None, index_col=0)

pca = PCA(n_components=2)
pca_model = pca.fit_transform(df_new)
pca_model = pd.DataFrame(pca_model)
```

Figure 7

```
sns_plot = sns.cubehelix_palette(light=1, as_cmap=True)
ax = sns.scatterplot(x=-pca_model[0], y=pca_model[1],
                     size=df.Eat,hue=df.Eat,
                     palette=sns_plot, sizes=(10, 60))
plt.xlabel('pca1')
plt.ylabel('pca2')
plt.title('Energy as function of two pca components');
```

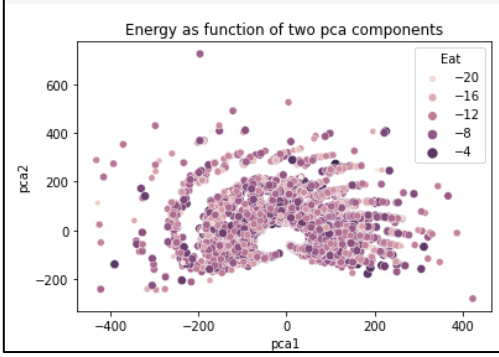

Figure 8

Figure 8 shows the plot of how the values are distributed for 2 different PCA component.

Below is the code for data preprocessing and feature engineering step

```
#import Libraries

import numpy as np
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA

from sklearn.model_selection import train_test_s
plit, GridSearchCV, RandomizedSearchCV
from sklearn.base import BaseEstimator, Transfo
rmerMixin, RegressorMixin

from sklearn.ensemble import RandomForestReg
ressor
from sklearn.neighbors import KNeighborsRegre
ssor
from sklearn.linear_model import Ridge, Lasso
from lightgbm.sklearn import LGBMRegressor
from xgboost.sklearn import XGBRegressor
from sklearn.metrics import mean_squared_error

from mlxtend.regressor import StackingCVRegre
ssor
from sklearn.model_selection import cross_val_s
core
from sklearn.metrics import make_scorer

#importing dataset
df = pd.read_csv('roboBohr.csv')
df.head()

print(df.shape)
df.isnull().sum().sum()

df.columns
df = df.drop(['Unnamed: 0', 'pubchem_id'], axis =
 1)
```

```
#shape and size of dataset and checking the null values
print(df.shape)
df.isnull().sum().sum()

#Checking the column names and dropping unwanted
df.columns
df = df.drop(['Unnamed: 0', 'pubchem_id'], axis = 1)

#Exploring Target variabel
df.Eat.describe()

sns.distplot(df['Eat'], kde=True)
plt.xlabel('Energy')
plt.ylabel('Frequency')
plt.title('Energy Distribution');

#Prinicple component Analysis with another dataset
df_new = pd.read_csv('coulombL.csv',header=None, index_col=0)

pca = PCA(n_components=2)
pca_model = pca.fit_transform(df_new)
pca_model = pd.DataFrame(pca_model)

sns_plot = sns.cubehelix_palette(light=1, as_cmap=True)
ax = sns.scatterplot(x=-pca_model[0], y=pca_model[1],
                     size=df.Eat,hue=df.Eat,
                     palette=sns_plot, sizes=(10, 60))
plt.xlabel('pca1')
plt.ylabel('pca2')
plt.title('Energy as function of two pca components');
```

IV. Model Development

In this step, we will be training different models on the pre-processed data.

We will create a dataset that is not having a target variable.

```
Y = df['Eat']
df = df.drop(['Eat'], axis = 1)
```

Figure 9

Splitting the data set into training and testing, for training we are using 75% of the data and 25% of the data is for testing

```
x_train, x_test, y_train, y_test = train_test_split(df, Y, test_size=0.25)
```

Figure 10

Defining model parameters for each model, here we have LGBM regressor, XG Boost, Random Forest, KNN, Ridge, Lasso, and Elastic net.

```
lgbm_model = LGBMRegressor(num_leaves = 20, n_estimators = 200, min_child_weight = 7,
                           max_depth = 5, learning_rate = 0.1,colsample_bytree = 0.3)

xgb_model = XGBRegressor(objective='reg:linear', eval_metric = 'rmse',
                         learning_rate = 0.0625, reg_lambda = 0,
                         max_depth = 5, colsample_bytree = 0.1,
                         min_child_weight = 12, n_estimators = 500)
rf_model = RandomForestRegressor(n_estimators = 125, min_samples_split = 5,
                                 max_features = 'auto', max_depth = 5)
knn_model = KNeighborsRegressor(weights = 'distance', n_neighbors = 5,
                                leaf_size = 100)
ridge_model = Ridge(alpha = 800)
lasso_model = Lasso(alpha = 0.05, max_iter=8000)
```

Figure 11

We have written a class to find the average of two models and use it as one. Here I am using GB and LGBT as two models to get the average.

```
class Regressor_Average_Score(BaseEstimator, RegressorMixin, TransformerMixin):
    def __init__(self, reg):
        self.reg = reg
        self.predict = None

    def fit(self, X, y):
        for regs in self.reg:
            regs.fit(X, y)
        return self

    def predict(self, X):
        self.predict = np.column_stack([regs.predict(X) for regs in self.reg])
        return np.mean(self.predict, axis=1)


averaged_model = Regressor_Average_Score([xgb_model, lgbm_model])
```

Figure 12

The below figure shows the root mean square error values for each model and calls each model for cross-validation to get the correct parameters for each model and then find the RMSE value for each model.

```
def root_mean_square(predicted, actual):
    return np.sqrt(np.mean(np.square(predicted - actual)))

rmse = make_scorer(root_mean_square, greater_is_better=False)

models = [
     ('RandomForest', rf_model),
     ('XGBoost', xgb_model),
     ('KNN', knn_model),
     ('Ridge', ridge_model),
     ('Lasso', lasso_model),
     ('LightGBM', lgbm_model),
     ('Averaged', averaged_model),
]

scores = [
    -1.0 * cross_val_score(model, x_train.values, y_train.values, scoring=rmse, cv=5).mean()
    for _,model in models
]

df_new = pd.DataFrame({ 'Model': [name for name, _ in models], 'Error (RMSE)': scores })
df_new.plot(x='Model', kind='bar')
```

Figure 13

The below figure shows the bar graph of the root means square error for each model.

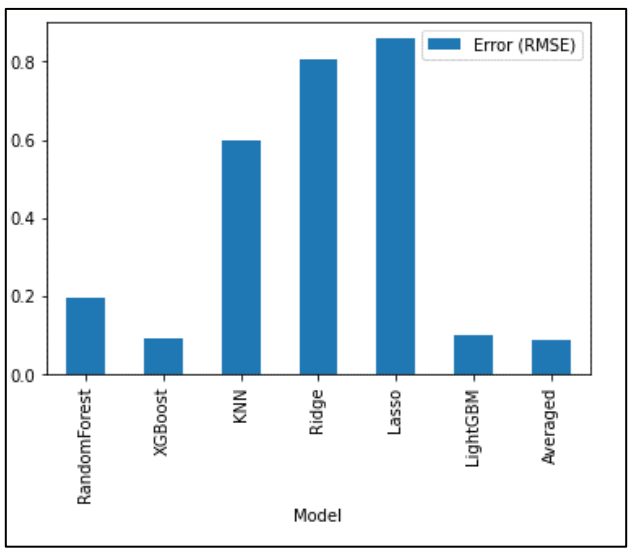

Figure 14

In figure 15, the table shows the root mean square value of each model.

df_new

	Model	Error (RMSE)
0	RandomForest	0.195093
1	XGBoost	0.090946
2	KNN	0.600281
3	Ridge	0.807118
4	Lasso	0.859679
5	LightGBM	0.099845
6	Averaged	0.088244

Figure 15

From the above table, Random Forest, XG Boost, Light GBM and averaged model are having least error values out of all.

Hence now we will train a model for the dataset for these models and predict for testing data.

Training the XG Boost model for training data and testing the model. Printing the RMSE value for the predicted value.

```
xgb_model.fit(x_train, y_train)
y_pred_xgb = xgb_model.predict(x_test)
print("RMSE Value for testing XGB model is "
          +str(np.sqrt(mean_squared_error(y_test, y_pred_xgb))))

[08:55:51] WARNING: /workspace/src/objective/regression_obj.cu:152
The score for xgb model is 0.08738262585311009
```

Figure 16

Training the Light GBM model for training data and testing the model. Printing the RMSE value for the predicted value.

```
lgbm_model.fit(x_train, y_train)
y_pred_lgbm = lgbm_model.predict(x_test)
print("RMSE Value for testing LightGBM model is "
          +str(np.sqrt(mean_squared_error(y_test, y_pred_lgbm))))

RMSE Value for testing LightGBM model is 0.09719250888816772
```

Figure 17

Training the Averaged Model for training data and testing the model. Printing the RMSE value for the predicted value.

```
averaged_model.fit(x_train, y_train)
y_pred_avg = averaged_model.predict(x_test)
print("RMSE Value for testing Averaged model (xgb + lgb) is "
          +str(np.sqrt(mean_squared_error(y_test, y_pred_avg))))

[09:19:14] WARNING: /workspace/src/objective/regression_obj.cu:152: reg:lin
RMSE Value for testing Averaged model (xgb + lgb) is 0.08548309692940105
```

Figure 18

Training the Random Forest Classifier model for training data and testing the model. Printing the RMSE value for the predicted value.

```
rf_model.fit(x_train, y_train)
y_pred_rf = rf_model.predict(x_test)
print("RMSE Value for testing Random Forest model is "
            +str(np.sqrt(mean_squared_error(y_test, y_pred_rf))))

RMSE Value for testing Random Forest model is 0.2001663493447642
```

Figure 19

Calculating and printing the mean value of the predicted value of each model and the mean value of test data.

```
print('XGBoost model test mean: %.3f' % y_pred_xgb.mean())
print('lgbm model test mean: %.3f' % y_pred_lgbm.mean())
print('Averaged model Test mean: %.3f' % y_pred_avg.mean())
print('Random Forest mean: %.3f' % y_pred_rf.mean())
print('Test mean: %.3f' % y_test.mean())

XGBoost model test mean: -11.214
lgbm model test mean: -11.214
Averaged model Test mean: -11.214
Random Forest mean: -11.211
Test mean: -11.212
```

Figure 20

Plotting the graph for each model validation for predicted value.

```
fig, ax = plt.subplots(nrows=5, sharex=True, sharey=True, figsize=(10,10))
sns.distplot(y_pred_xgb, ax=ax[0], color='red', label='XGBoost validation xgb')
sns.distplot(y_pred_lgbm, ax=ax[1], color='green', label='Light GBM validation')
sns.distplot(y_pred_avg, ax=ax[2], color='yellow', label='Averaged model validation')
sns.distplot(y_pred_rf, ax=ax[3], color='orange', label='Random Forest validation')
sns.distplot(y_test, ax=ax[4], color='blue', label='test')

ax[0].legend(loc=0)
ax[1].legend(loc=0)
ax[2].legend(loc=0)
ax[3].legend(loc=0)
ax[4].legend(loc=0)
plt.show()
```

Figure 21

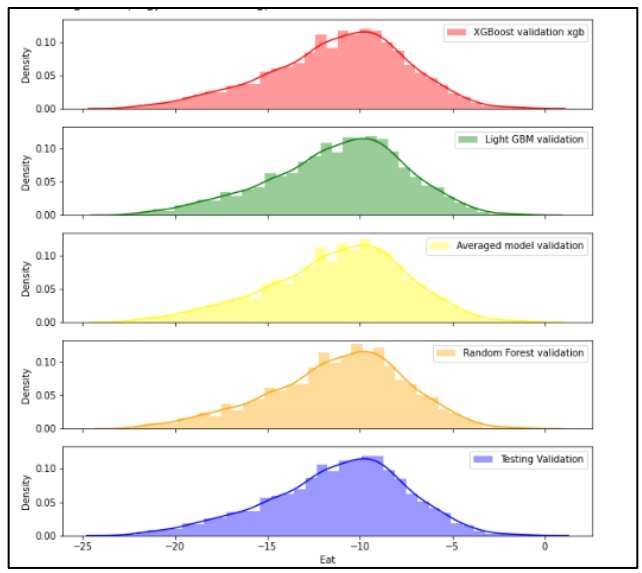

Figure 22

Below is the code for model development and testing.

#Creating target variable and other features.
Y = df['Eat']
df = df.drop(['Eat'], axis = 1)
# Splitting dataset into Training and testing dataset

x_train, x_test, y_train, y_test = train_test_split(df, Y, test_size=0.25)

```
#Defining parameters for each model
lgbm_model = LGBMRegressor(num_leaves = 2
0, n_estimators = 200, min_child_weight = 7, ma
x_depth = 5, learning_rate = 0.1,colsample_bytre
e = 0.3)

xgb_model = XGBRegressor(objective='reg:line
ar', eval_metric = 'RMSE', learning_rate = 0.062
5, reg_lambda = 0,max_depth = 5, colsample_by
tree = 0.1, min_child_weight = 12, n_estimators
= 500)

rf_model = RandomForestRegressor(n_estimator
s = 125, min_samples_split = 5, max_features = '
auto', max_depth = 5)

knn_model = KNeighborsRegressor(weights = 'd
istance', n_neighbors = 5, leaf_size = 100)

ridge_model = Ridge(alpha = 800)

lasso_model = Lasso(alpha = 0.05, max_iter=800
0)

#Class to merge two model

class Regressor_Average_Score(BaseEstimator,
RegressorMixin, TransformerMixin):

    def __init__(self, reg):
        self.reg = reg
        self.predict = None

    def fit(self, X, y):
        for regs in self.reg:
            regs.fit(X, y)
        return self

    def predict(self, X):
        self.predict = np.column_stack([regs.predict
(X) for regs in self.reg])
        return np.mean(self.predict, axis=1)

#Function for Root mean square value.
def root_mean_square(predicted, actual):
    return np.sqrt(np.mean(np.square(predicted - a
ctual)))

RMSE = make_scorer(root_mean_square, greate
r_is_better=False)

models = [
    ('RandomForest', rf_model),
    ('XGBoost', xgb_model),
    ('KNN', knn_model),
    ('Ridge', ridge_model),
    ('Lasso', lasso_model),
    ('LightGBM', lgbm_model),
    ('Averaged', averaged_model),
]

scores = [
    -
1.0 * cross_val_score(model, x_train.values, y_tr
ain.values, scoring=RMSE, cv=5).mean()
    for _,model in models
]

df_new = pd.DataFrame({ 'Model': [name for na
me, _ in models], 'Error (RMSE)': scores })
df_new.plot(x='Model', kind='bar')

df_new

#Training and fitting each model and then
predicting

averaged_model.fit(x_train, y_train)
y_pred_avg = averaged_model.predict(x_test)
print("RMSE Value for testing Averaged model (
xgb + lgb) is "+str(np.sqrt(mean_squared_error(y
_test, y_pred_avg))))

xgb_model.fit(x_train, y_train)
y_pred_xgb = xgb_model.predict(x_test)
print("RMSE Value for testing XGB model is "+
str(np.sqrt(mean_squared_error(y_test, y_pred_x
gb))))
```

```
plt.show()rf_model.fit(x_train, y_train)
y_pred_rf = rf_model.predict(x_test)
print("RMSE Value for testing Random Forest m
odel is "+str(np.sqrt(mean_squared_error(y_test,
y_pred_rf))))

lgbm_model.fit(x_train, y_train)
y_pred_lgbm = lgbm_model.predict(x_test)
print("RMSE Value for testing LightGBM model
 is "+str(np.sqrt(mean_squared_error(y_test, y_pr
ed_lgbm))))

print('XGBoost model test mean: %.3f' % y_pred
_xgb.mean())
print('lgbm model test mean: %.3f' % y_pred_lgb
m.mean())
print('Averaged model Test mean: %.3f' % y_pre
d_avg.mean())
print('Random Forest mean: %.3f' % y_pred_rf.
mean())
print('Test mean: %.3f' % y_test.mean())

#Plotting test value error validation

fig, ax = plt.subplots(nrows=5, sharex=True, shar
ey=True, figsize=(10,10))
sns.distplot(y_pred_xgb, ax=ax[0], color='red', la
bel='XGBoost validation xgb')
sns.distplot(y_pred_lgbm, ax=ax[1], color='green
', label='Light GBM validation')
sns.distplot(y_pred_avg, ax=ax[2], color='yellow
', label='Averaged model validation')
sns.distplot(y_pred_rf, ax=ax[3], color='orange',
label='Random Forest validation')
sns.distplot(y_test, ax=ax[4], color='blue', label='
Testing Validation')

ax[0].legend(loc=0)
ax[1].legend(loc=0)
ax[2].legend(loc=0)
ax[3].legend(loc=0)
ax[4].legend(loc=0)
```

V. Conclusion

From the 4 trained models, the best error rate value for the training and testing dataset is seen for an averaged model which is a combination of two models (XG Boost and LGBM). For training, the RMSE value is 0.088244 and the testing RMSE value is 0.085483. Hence averaged model can be used for the prediction of energy from the molecular state.

VI. References

I. Yixiao Chen,Linfeng Zhang,Han Wang , Weinan E, “Ground State Energy Functional with Hartree–Fock Efficiency and Chemical Accuracy”

II. Akshaya Karthikeya, U Deva Priyakumar,

III. “Artificial intelligence: machine learning for chemical sciences”

IV. Yenwee Lim, “Model Averaging” , https://towardsdatascience.com/ensemble-averaging-improve-machine-learning-performance-by-voting-246106c753ee

V. “Scikit learn” , https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.train_test_split.html

VI. Datamite, “PCA analysis”, https://datamites.com/blog/a-guide-to-principal-component-analysis-pca-for-machine-learning/

VII. Elluru Pavan Kumar Reddy, “Feature Engineering Steps”, https://www.analyticsvidhya.com/blog/2021/03/step-by-step-process-of-feature-engineering-for-machine-learning-algorithms-in-data-science/

VIII. “Seaborn sns plotting” https://seaborn.pydata.org/tutorial/introduction

IX. Ben Pilkington, “Use of AI in chemical”, https://www.azom.com/article.aspx?ArticleID=21998